

What is Bucket Sort?

Bucket Sort is a sorting algorithm that:

- Divides the data into several buckets (groups).
- Places each element into its appropriate bucket.
- Sorts each bucket individually.
- Concatenates all buckets.

Unlike Merge Sort or Quick Sort, Bucket Sort tries to reduce the sorting work by grouping similar values together first.

Real-Life Analogy

Imagine you have to sort exam papers based on marks.

Students scored:

91 23 45 87 76 54 32 15 98 67

Instead of sorting all 10 numbers together, you first make groups:

0-9
10-19
20-29
30-39
40-49
50-59
60-69
70-79
80-89
90-99

Now place every student into the correct group.

Group	Elements
0-9	Empty
10-19	15
20-29	23
30-39	32
40-49	45
50-59	54
60-69	67
70-79	76
80-89	87
90-99	91 98

Now sort each bucket separately.

Most buckets contain only one element, so almost no work is needed.

Finally, collect buckets from left to right.

15
23
32
45
54
67
76
87
91
98

Array sorted.

Basic Algorithm

Suppose

arr = [29, 25, 3, 49, 9, 37, 21, 43]

Step 1

Create buckets.
For example

Bucket0	0-9
Bucket1	10-19
Bucket2	20-29
Bucket3	30-39
Bucket4	40-49

Step 2

Insert elements.

29 ↓ Bucket2
9 ↓ Bucket0

25 ↓ Bucket2
37 ↓ Bucket3

3 ↓ Bucket0
21 ↓ Bucket2

49 ↓ Bucket4
43 ↓ Bucket4

Now buckets look like

Bucket0	3 9
Bucket1	Empty
Bucket2	29 25 21
Bucket3	37
Bucket4	49 43

Notice

Each bucket is not sorted yet.

Step 3

Sort every bucket.

Bucket0

3 9
Already sorted.

Bucket2

29 25 21
Sort it
21 25 29

Bucket4

49 43
Sort
43 49

Step 4

Merge buckets.

Bucket0 3 9 ↓ Bucket1 Empty ↓ Bucket2 21 25 29 ↓ Bucket3 37 ↓ Bucket4 43 49

Final answer

3 9 21 25 29 37 43 49

Sorted.

Visualization

Original

29 25 3 49 9 37 21 43



Buckets

Bucket0	3 9
Bucket1	-
Bucket2	29 25 21
Bucket3	37
Bucket4	49 43



Sort inside buckets

3 9 | 21 25 29 | 37 | 43 49



Merge

3 9 21 25 29 37 43 49

General Bucket Sort Algorithm

1. Find minimum and maximum value (optional, depending on implementation).
2. Decide how many buckets to create.
3. Create empty buckets.
4. For each element:
 - Find its bucket.
 - Insert it.
5. Sort each bucket.
6. Merge all buckets.
7. Sorted array obtained.

How do we find the bucket?

Usually

```
bucketIndex = (value - minValue) / bucketSize;
```

Example

Min = 0 | Bucket Size = 10 | Value = 25 | Bucket = $25/10 = 2$ → So **Bucket2**

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void bucketSort(vector<int>& arr) {
    int n = arr.size();
    int mx = *max_element(arr.begin(), arr.end());
    int bucketSize = 10;
    int bucketCount = mx / bucketSize + 1;
    vector<vector<int>> buckets(bucketCount);

    for(int x : arr){
        int idx = x / bucketSize;
        buckets[idx].push_back(x);
    }

    arr.clear();
    for(auto &bucket : buckets){
        sort(bucket.begin(), bucket.end());
        for(int x : bucket)
            arr.push_back(x);
    }
}

int main(){
    vector<int> arr={29,25,3,49,9,37,21,43};
    bucketSort(arr);
    for(int x:arr)
        cout<<x<<" ";
}
```

Output

3 9 21 25 29 37 43 49

Time Complexity

Best Case

Every bucket contains one element.

Insertion: $O(n)$

Sorting buckets: $O(1)$

Overall: $O(n)$

Average Case

Elements are evenly distributed.

Each bucket has a few elements.

Overall: $O(n)$

Worst Case

Everything goes into one bucket.

sort(one bucket) = $O(n \log n)$

Overall: $O(n \log n)$

Space Complexity

Need buckets. $O(n)$

When does Bucket Sort work well?

Bucket Sort is most effective when the input values are uniformly distributed across a known range.

Examples:

- Student marks (0–100)
- Ages (0–120)
- Decimal numbers in the range [0, 1)
- Floating-point values that are evenly spread
- Scores, percentages, ratings

When should you avoid Bucket Sort?

Bucket Sort performs poorly when:

- Data is highly skewed (most values fall into one bucket).
- The value range is extremely large compared to the number of elements.
- You don't have a reasonable way to choose bucket sizes.

Example:

1 2 3 4 1000000000

Most buckets would be empty or one bucket would dominate, making Bucket Sort inefficient.

Bucket Sort vs Counting Sort vs Radix Sort

Feature	Bucket Sort	Counting Sort	Radix Sort
Uses buckets?	✓ Yes	✗ No	✗ No
Sorts inside buckets?	✓ Yes	✗ No	✗ No (uses Counting Sort repeatedly)
Works for large values?	✓ Yes (if distributed well)	✗ Only for small ranges	✓ Yes
Comparison sort inside?	Usually sort() or Insertion Sort	No	No
Best Time	$O(n)$	$O(n + k)$	$O(d \times n)$
Worst Time	$O(n \log n)$	$O(n + k)$	$O(d \times n)$

Interview Summary

A good interview explanation is:

Bucket Sort is a distribution-based sorting algorithm. It divides the input into multiple buckets based on value ranges, sorts each bucket individually (often using insertion sort or `std::sort`), and then concatenates the buckets to produce the final sorted array. Its average-case time complexity is $O(n)$ when the data is uniformly distributed, but in the worst case, if many elements fall into the same bucket, it degrades to $O(n \log n)$ because one bucket must be sorted conventionally.

This is the general Bucket Sort algorithm. The Maximum Gap problem borrows the idea of buckets but does not sort inside the buckets. Instead, it stores only the minimum and maximum value of each bucket because the goal is to find the largest adjacent gap, not to fully sort the array. That's why the Maximum Gap solution remains $O(n)$ in all cases.